

Practical Number		CO/PO
	Post Laboratory Work Description: Commit and tag lab-09. Record the wrong balance before synchronization and the correct balance after, and explain the difference. Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Working multithreaded deposits – 30%; demonstrating the race condition – 25%; correct synchronization fix – 30%; viva – 15%. Expected Input / Process / Output Input: In code: 10 threads, each calling deposit(1) one thousand times, on one account starting at balance 0. Process: Without synchronization the threads overwrite each other's updates; with synchronization each update completes before the next begins. Expected Output: Before the fix the printed balance is less than 10000 and varies each run; after the fix it is exactly 10000 every run.	

Practical Number		CO/PO
10	Part A — Practice Programs (Hour 1) Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Thread-pool runner: with a fixed pool of 3 (ExecutorService), submit about 10 Runnable tasks that print their id and sleep briefly; shut down and await completion, observing the few threads being reused. 2. Producer–consumer: one thread adds numbers to a small shared buffer and another removes them, coordinated with wait()/notify(); produce and consume a fixed number of items in order with none lost. 3. (For Advanced Learners – home) Reproduce a deadlock between two threads that lock two objects in opposite order, then remove it with a consistent lock order. Choose the scenario yourself. Part B — Project Work (Hour 2): MiniBank Building on basic threads, this lab gives MiniBank a proper transaction engine. You process a batch of transactions on a managed thread pool, coordinate two threads with wait/notify, and learn to avoid deadlock with consistent lock ordering — the way back-office systems process many transactions safely. 1. Create a class named TransactionProcessor that holds an ExecutorService created with Executors.newFixedThreadPool(4). Add a method submit(Runnable task) that calls execute(), and a method stop() that calls shutdown() and awaitTermination(). 2. Create Runnable tasks that each perform a deposit or withdrawal; submit many of them, then stop the processor and wait for completion. 3. Build a simple producer-consumer: one thread adds transactions to a shared array buffer and another thread processes them, using wait() and notify() inside synchronized blocks. 4. Create a deadlock: two threads transfer money A to B and B to A, locking the two accounts in opposite order. Then fix it by always locking the account with the smaller accountNumber first. Key Questions / Analysis / Interpretation to be evaluated during/after Implementation 1. Why is a thread pool better than creating a new thread per task? 2. How do wait() and notify() coordinate two threads? 3. What causes a deadlock, and how does consistent lock ordering prevent it? Supplementary Problems – • Print how many tasks each pool thread executed. • Detect a long-running transfer and log a warning. Key Skills to be addressed – ExecutorService and thread pools; submitting Runnable tasks; inter-thread communication with wait/notify; deadlock and lock ordering. Applications – Batch processors, web-server request pools and job-scheduling systems all use managed thread pools.	CO4 PO5, PO11, PSO1

Practical Number		CO/PO
	Learning Outcome – The student can run tasks on a thread pool, coordinate threads, and avoid deadlock. (CO4) Dataset / Test Data (Source and Description, if applicable): No external dataset. A batch of transactions is created in code and submitted to the pool. Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git. • Total Hours of Problem Definition Implementation: 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • Total Hours of Engagement = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours. Post Laboratory Work Description: Commit and tag lab-10. Demonstrate the deadlock first, then show the fixed version completing normally. Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Thread-pool processor – 30%; producer-consumer with wait/notify – 25%; deadlock created and fixed – 30%; viva – 15%. Expected Input / Process / Output Input: In code: two threads, one transferring from account A to B and one from B to A, run with both lock orders. Process: With opposite lock orders each thread holds one account and waits for the other (deadlock); with the lower-number-first rule one thread completes before the other starts. Expected Output: The opposite-order version hangs (deadlock); the fixed version finishes and both balances are correct.	

Practical Number		CO/PO
11	Part A — Practice Programs (Hour 1) Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Save/load: make a small class Serializable, save an array of its objects to a file with ObjectOutputStream, then read it back into fresh objects and confirm the values survived. Mark one field transient and show it is not saved. 2. Log analyzer: create a few text files, then use Path/Files to read each line by line and count the total lines and how many contain a keyword across all files; also print each file's size from its attributes. 3. (For Advanced Learners – home) Walk a folder tree with NIO and produce a report of the files it contains (name, size, modified time), writing the summary to a file; decide what to include. Part B — Project Work (Hour 2): MiniBank So far MiniBank forgets everything when it closes. In this lab you add the persistence and reporting layer: you make the model Serializable and save/load all accounts to a file, write an append-only transaction log, and then use NIO to walk the logs and produce an end-of-day summary report. 1. Make Account (and its subclasses and Customer) implement Serializable, and add a private static final long serialVersionUID field. 2. Create a class named StatePersister with a public static method void save(Account[] accounts, Path file) that uses ObjectOutputStream inside try-with-resources, and a public static method Account[] load(Path file) that uses ObjectInputStream. 3. Create a class named TransactionLog with a public static method void append(Path file, String line) that uses Files.write(...) with CREATE and APPEND. 4. Create a class named ReportGenerator that walks the logs folder using Files.newDirectoryStream, reads each file line by line with a BufferedReader, and adds	CO5 PO4, PO5, PSO1, PSO2